ASSIGNMENT-12.1

Name: M.Rohan

HT. No: 2303A51545

Batch: 08

Task Description #1 (Sorting – Merge Sort Implementation)

• Task: Use AI to generate a Python program that implements

the Merge Sort algorithm.

• Instructions:

o Prompt AI to create a function merge_sort(arr) that

sorts a list in ascending order.

o Ask AI to include time complexity and space

complexity in the function docstring. Verify

the generated code with test cases

Lab_12.py > merge_sort

```python
# Generate a Python program that implements Merge Sort.
# Create a function merge_sort(arr) that sorts a list in ascending order.
# Include time complexity and space complexity in the function docstring.
# Add a few test cases to verify correctness.
def merge_sort(arr):
    """
    Sorts a list in ascending order using the Merge Sort algorithm.
    Time Complexity: O(n log n) - where n is the number of elements in the list.
    Space Complexity: O(n) - due to the temporary arrays used for merging.
    Parameters:
    arr (list): The list to be sorted.
    Returns:
    list: A new sorted list in ascending order.
    """
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid])
    right_half = merge_sort(arr[mid:])
    return merge(left_half, right_half)
def merge(left, right):
    """Merges two sorted lists into a single sorted list."""
    merged = []
    left_index = right_index = 0
    while left_index < len(left) and right_index < len(right):
        if left[left_index] < right[right_index]:
            merged.append(left[left_index])
            left_index += 1
        else:
            merged.append(right[right_index])
            right_index += 1
    # If there are remaining elements in left or right, add them to merged
    merged.extend(left[left_index:])
    merged.extend(right[right_index:])
    return merged
# Test cases
print(merge_sort([38, 27, 43, 3, 9, 82, 10]))  # Expected output: [3, 9, 10, 27, 38, 43, 82]
print(merge_sort([]))  # Expected output: []
print(merge_sort([5]))  # Expected output: [5]
print(merge_sort([5, 2, 9, 1, 5, 6]))  # Expected output: [1, 2, 5, 5, 6, 9]
print(merge_sort([3, 0, 2, 5, -1, 4, 1]))  # Expected output: [-1, 0, 1, 2, 3, 4, 5]
```

Task Description #2 (Searching – Binary Search with AI Optimization)

• Task: Use AI to create a binary search function that finds a target element in a sorted list.

• Instructions:

o Prompt AI to create a function binary_search(arr, target) returning the index of the target or -1 if not found.

o Include docstrings explaining best, average, and

worst-case complexities.

o Test with various inputs.

```python
# Generate a Python function binary_search(arr, target) that returns the index of the target in a sorted list or -1 if not found.
# Include a docstring with best, average, and worst-case time complexities.
# Add multiple test cases to verify the function.
def binary_search(arr, target):
    """
    Performs binary search on a sorted list to find the index of the target element.
    Time Complexity:
    - Best Case: O(1) - when the target is at the middle of the list.
    - Average Case: O(log n) - where n is the number of elements in the list.
    - Worst Case: O(log n) - when the target is not found or is at the end of the list.

    Parameters:
    arr (list): A sorted list in which to search for the target.
    target: The element to search for in the list.

    Returns:
    int: The index of the target if found, otherwise -1.
    """
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = left + (right - left) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
# Test cases
print(binary_search([1, 2, 3, 4, 5], 3))  # Expected output: 2
print(binary_search([1, 2, 3, 4, 5], 6))  # Expected output: -1
print(binary_search([], 1))  # Expected output: -1
print(binary_search([1], 1))  # Expected output: 0
print(binary_search([1, 2, 3, 4, 5], 1))  # Expected output: 0
print(binary_search([1, 2, 3, 4, 5], 5))  # Expected output: 4
```

Task Description #3 (Real-Time Application – Inventory

Management System)

• Scenario: A retail store’s inventory system contains

thousands of products, each with attributes like product ID,

name, price, and stock quantity. Store staff need to:

1. Quickly search for a product by ID or name.

2. Sort products by price or quantity for stock analysis.

• Task:

o Use AI to suggest the most efficient search and sort

algorithms for this use case.

o Implement the recommended algorithms in Python.

o Justify the choice based on dataset size, update

frequency, and performance requirements.

```python
# You are building an inventory management system with thousands of products (product_id, name, price, stock).
# 1. Suggest the most efficient search and sorting algorithms for this scenario.
# 2. Implement them in Python to:
#    - search by product ID or name
#    - sort by price and stock quantity
# 3. Justify the algorithm choices based on dataset size, update frequency, and performance needs.
# 1. For searching by product ID or name, a hash table (dictionary in Python) would be the most efficient data structure,
# as it provides average O(1) time complexity for lookups. For sorting by price and stock quantity,
# the Timsort algorithm (used by Python's built-in sorted() function) is efficient
# for real-world data and has a time complexity of O(n log n) in the worst case.
# 2. Implementation:
class Product:
    def __init__(self, product_id, name, price, stock):
        self.product_id = product_id
        self.name = name
        self.price = price
        self.stock = stock
class Inventory:
    def __init__(self):
        self.products_by_id = {}
        self.products_by_name = {}

    def add_product(self, product):
        self.products_by_id[product.product_id] = product
        self.products_by_name[product.name] = product

    def search_by_id(self, product_id):
        return self.products_by_id.get(product_id, None)

    def search_by_name(self, name):
        return self.products_by_name.get(name, None)

    def sort_by_price(self):
        return sorted(self.products_by_id.values(), key=lambda x: x.price)

    def sort_by_stock(self):
        return sorted(self.products_by_id.values(), key=lambda x: x.stock)
# Test cases
inventory = Inventory()
inventory.add_product(Product(1, "Laptop", 999.99, 10))
inventory.add_product(Product(2, "Smartphone", 499.99, 50))
inventory.add_product(Product(3, "Headphones", 199.99, 100))
# Search by product ID
```

```python
Click to add a breakpoint
inventory = Inventory()
inventory.add_product(Product(1, "Laptop", 999.99, 10))
inventory.add_product(Product(2, "Smartphone", 499.99, 50))
inventory.add_product(Product(3, "Headphones", 199.99, 100))
# Search by product ID
print(inventory.search_by_id(1).name)  # Expected output: "Laptop"
print(inventory.search_by_id(4))  # Expected output: None
# Search by product name
print(inventory.search_by_name("Smartphone").price)  # Expected output: 499.99
print(inventory.search_by_name("Tablet"))  # Expected output: None
# Sort by price
sorted_by_price = inventory.sort_by_price()
print([product.name for product in sorted_by_price])  # Expected output: ["Headphones", "Smartphone", "Laptop"]
# Sort by stock
sorted_by_stock = inventory.sort_by_stock()
print([product.name for product in sorted_by_stock])  # Expected output: ["Laptop", "Smartphone", "Headphones"]
# Justification: The use of a hash table allows for efficient searching by product ID and name,
# which is crucial for an inventory management system with thousands of products.
# The Timsort algorithm is efficient for sorting large datasets and is optimized for real-world data,
#  making it suitable for sorting by price and stock quantity. This approach ensures that the system can handle frequent updates
# and provide fast search and sort operations, meeting performance needs effectively.
```

Task description #4: Smart Hospital Patient Management

System

A hospital maintains records of thousands of patients with details

such as patient ID, name, severity level, admission date, and bill

amount. Doctors and staff need to:

1. Quickly search patient records using patient ID or name.

2. Sort patients based on severity level or bill amount for prioritization and billing.

Student Task

• Use AI to recommend suitable searching and sorting algorithms.

• Justify the selected algorithms in terms of efficiency and suitability.

• Implement the recommended algorithms in Python.

```python
# You are designing a Smart Hospital Patient Management System with thousands of records (patient_id, name, severity, admission_date, bill_amount).
# 1. Recommend suitable searching and sorting algorithms.
# 2. Justify the choices based on efficiency and real-time usage.
# 3. Implement the algorithms in Python to:
#    - search by patient ID or name
#    - sort by severity level and bill amount.
# 1. For searching by patient ID or name, a hash table (dictionary in Python) is recommended for its average O(1) time complexity for lookups.
# For sorting by severity level and bill amount, the Timsort algorithm (used by Python's built-in sorted() function) is efficient
# for real-world data and has a time complexity of O(n log n) in the worst case.
# 2. The use of a hash table allows for efficient searching by patient ID and name, which is crucial for a patient management system with thousands of records.
#  The Timsort algorithm is optimized for real-world data and is suitable for sorting by severity level and bill amount,
# ensuring that the system can handle frequent updates and provide fast search and sort operations, meeting performance needs effectively.
# 3. Implementation:
class Patient:
    def __init__(self, patient_id, name, severity, admission_date, bill_amount):
        self.patient_id = patient_id
        self.name = name
        self.severity = severity
        self.admission_date = admission_date
        self.bill_amount = bill_amount
class PatientManagementSystem:
    def __init__(self):
        self.patients_by_id = {}
        self.patients_by_name = {}

    def add_patient(self, patient):
        self.patients_by_id[patient.patient_id] = patient
        self.patients_by_name[patient.name] = patient

    def search_by_id(self, patient_id):
        return self.patients_by_id.get(patient_id, None)

    def search_by_name(self, name):
        return self.patients_by_name.get(name, None)

    def sort_by_severity(self):
        return sorted(self.patients_by_id.values(), key=lambda x: x.severity)

    def sort_by_bill_amount(self):
        return sorted(self.patients_by_id.values(), key=lambda x: x.bill_amount)
# Test cases
pms = PatientManagementSystem()
pms.add_patient(Patient(1, "John Doe", 5, "2024-01-01", 1000.00))
```

```python
        return sorted(self.patients_by_id.values(), key=lambda x: x.bill_amount)
# Test cases
pms = PatientManagementSystem()
pms.add_patient(Patient(1, "John Doe", 5, "2024-01-01", 1000.00))
pms.add_patient(Patient(2, "Jane Smith", 3, "2024-02-01", 500.00))
pms.add_patient(Patient(3, "Alice Johnson", 4, "2024-03-01", 750.00))
# Search by patient ID
print(pms.search_by_id(1).name)  # Expected output: "John Doe"
print(pms.search_by_id(4))  # Expected output: None
# Search by patient name
print(pms.search_by_name("Jane Smith").bill_amount)  # Expected output: 500.00
print(pms.search_by_name("Bob Brown"))  # Expected output: None
# Sort by severity
sorted_by_severity = pms.sort_by_severity()
print([patient.name for patient in sorted_by_severity])  # Expected output: ["Jane Smith", "Alice Johnson", "John Doe"]
# Sort by bill amount
sorted_by_bill_amount = pms.sort_by_bill_amount()
print([patient.name for patient in sorted_by_bill_amount])  # Expected output: ["Jane Smith", "Alice Johnson", "John Doe"]
```

Task Description #5: University Examination Result Processing

System

A university processes examination results for thousands of students containing roll number, name, subject, and marks. The system must:

1. Search student results using roll number.

2. Sort students based on marks to generate rank lists.

Student Task

• Identify efficient searching and sorting algorithms using AI assistance.

• Justify the choice of algorithms.

• Implement the algorithms in Python.

```python
# You are building a University Examination Result Processing System with thousands of records (roll_no, name, subject, marks).
# 1. Recommend efficient searching and sorting algorithms.
# 2. Justify the choices based on dataset size and performance.
# 3. Implement Python code to:
#    - search student results by roll number
#    - sort students by marks to generate rank lists.
# 1. For searching student results by roll number, a hash table (dictionary in Python) is recommended for its average O(1) time complexity for lookups.
# For sorting students by marks to generate rank lists, the Timsort algorithm (used by Python's built-in sorted() function)
#  is efficient for real-world data and has a time complexity of O(n log n) in the worst case.
# 2. The use of a hash table allows for efficient searching by roll number, which is crucial for a result processing system with thousands of records.
# The Timsort algorithm is optimized for real-world data and is suitable for sorting by marks,
# ensuring that the system can handle frequent updates and provide fast search and sort operations, meeting performance needs effectively.
# 3. Implementation:
class StudentResult:
    def __init__(self, roll_no, name, subject, marks):
        self.roll_no = roll_no
        self.name = name
        self.subject = subject
        self.marks = marks
class ResultProcessingSystem:
    def __init__(self):
        self.results_by_roll_no = {}

    def add_result(self, result):
        self.results_by_roll_no[result.roll_no] = result

    def search_by_roll_no(self, roll_no):
        return self.results_by_roll_no.get(roll_no, None)

    def sort_by_marks(self):
        return sorted(self.results_by_roll_no.values(), key=lambda x: x.marks, reverse=True)
# Test cases
rps = ResultProcessingSystem()
rps.add_result(StudentResult(1, "John Doe", "Math", 85))
rps.add_result(StudentResult(2, "Jane Smith", "Math", 90))
rps.add_result(StudentResult(3, "Alice Johnson", "Math", 80))
# Search by roll number
print(rps.search_by_roll_no(1).name)  # Expected output: "John Doe"
print(rps.search_by_roll_no(4))  # Expected output: None
# Sort by marks to generate rank list
sorted_by_marks = rps.sort_by_marks()
print([(result.name, result.marks) for result in sorted_by_marks])  # Expected output: [("Jane Smith", 90), ("John Doe", 85), ("Alice Johnson", 80)]
```

Task Description #6: Online Food Delivery Platform

An online food delivery application stores thousands of orders with order ID, restaurant name, delivery time, price, and order status. The platform needs to:

1. Quickly find an order using order ID.

2. Sort orders based on delivery time or price.

Student Task

• Use AI to suggest optimized algorithms.

• Justify the algorithm selection.

• Implement searching and sorting modules in Python.

```python
# You are designing an Online Food Delivery Platform with thousands of orders (order_id, restaurant_name, delivery_time, price, order_status).
# 1. Suggest optimized searching and sorting algorithms.
# 2. Justify the algorithm choices based on efficiency and scalability.
# 3. Implement Python code to:
#    - search an order by order_id
#    - sort orders by delivery_time and price.
# 1. For searching an order by order_id, a hash table (dictionary in Python) is recommended for its average O(1) time complexity for lookups.
# For sorting orders by delivery_time and price, the Timsort algorithm (used by Python's built-in sorted() function) is
# efficient for real-world data and has a time complexity of O(n log n) in the worst case.
# 2. The use of a hash table allows for efficient searching by order_id, which is crucial for an online food delivery platform with thousands of orders.
# The Timsort algorithm is optimized for real-world data and is suitable for sorting by delivery_time and price,
# ensuring that the system can handle frequent updates and provide fast search and sort operations, meeting performance needs effectively.
# 3. Implementation:
class Order:
    def __init__(self, order_id, restaurant_name, delivery_time, price, order_status):
        self.order_id = order_id
        self.restaurant_name = restaurant_name
        self.delivery_time = delivery_time
        self.price = price
        self.order_status = order_status
class FoodDeliveryPlatform:
    def __init__(self):
        self.orders_by_id = {}
    def add_order(self, order):
        self.orders_by_id[order.order_id] = order
    def search_by_order_id(self, order_id):
        return self.orders_by_id.get(order_id, None)
    def sort_by_delivery_time(self):
        return sorted(self.orders_by_id.values(), key=lambda x: x.delivery_time)
    def sort_by_price(self):
        return sorted(self.orders_by_id.values(), key=lambda x: x.price)
# Test cases
platform = FoodDeliveryPlatform()
platform.add_order(Order(1, "Pizza Place", "2024-01-01 18:00", 20.00, "Delivered"))
platform.add_order(Order(2, "Sushi Spot", "2024-01-01 19:00", 30.00, "In Progress"))
platform.add_order(Order(3, "Burger Joint", "2024-01-01 17:30", 15.00, "Delivered"))
# Search by order ID
print(platform.search_by_order_id(1).restaurant_name)  # Expected output: "Pizza Place"
print(platform.search_by_order_id(4))  # Expected output: None
# Sort by delivery time
sorted_by_delivery_time = platform.sort_by_delivery_time()
print([(order.restaurant_name, order.delivery_time) for order in sorted_by_delivery_time])
 # Expected output: [("Burger Joint", "2024-01-01 17:30"), ("Pizza Place", "2024-01-01 18:00"), ("Sushi Spot", "2024-01-01 19:00")]
# Sort by price
sorted_by_price = platform.sort_by_price()
print([(order.restaurant_name, order.price) for order in sorted_by_price])  # Expected output: [("Burger Joint", 15.00), ("Pizza Place", 20.00), ("Sushi Spot", 30.00)]
```